Deliverable 5: AI Partnership Log

A candid record of how AI was used building this

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

AI Partnership Log

Which tools, for what

Claude Code (Claude Sonnet 5) was the primary collaborator for essentially the entire build: architecture, the data generator, the feature engineering, the anomaly detector, the backend API, the React frontend, the AI copilot layer, the Playwright suite, the CI config, and a first draft of every doc in this folder. I didn't hand-write code alongside it. I directed it, reviewed what it produced, and pushed back when something didn't sit right.

Separately, Claude (Haiku 4.5, via the Anthropic API) is *inside* the product itself: the "Explain this stand's status" copilot on the dashboard. That's a different relationship. It's not helping me build, it's a component I built, and I had to think about its failure modes the way I'd think about any other dependency, not trust it because it's the same underlying company and model family as my coding assistant.

Overrides, where I didn't just take what it gave me

1. Publishing the confidential prompt. I told Claude Code to "sync all the setup" to the public GitHub repo, meaning everything, including `assessment.md`, the literal take-home prompt. It flagged, unprompted, that the assessment document itself says not to post the prompt to public repos, and asked how I wanted to handle it before pushing anything. I hadn't thought about that when I gave the instruction; I was thinking about the code, not the prompt file sitting next to it. I told it to keep those files local-only. Small moment, but it's exactly the kind of "did you actually think about what you just told me to do" check I want, and it's the one override that came from the AI catching *me*, not the other way around.

2. Accepting the first "it works" without asking what it cost. First pass at the anomaly detector's alert threshold used the 99.5th percentile of normal scores, conservative-sounding, safe-sounding. It caught zero of eleven real failures in the held-out test. I didn't just take a fix; I had it sweep the full range of thresholds and show me the actual detection-rate versus false-alarm-rate tradeoff at each point, not just report back "fixed it, 11/11 now." Landed on the 97th percentile, full detection, 1.06% false-alarm rate, but I picked that point after seeing the tradeoff table, not because it was the first setting that produced a good headline number.

3. Documentation voice. Claude's default instinct for technical docs is polished, third-person, "the system was designed to..." corporate voice. I asked for the opposite: first-person, what I actually tried, what didn't work, what I felt when something clicked. That's not a style preference for its own sake. This assignment explicitly scores the AI Partnership Log on candor and specificity, and generic corporate-voice docs would have flattened months of real back-and-forth into something that reads like it was written by nobody in particular. I also had to push on this a second time: even after asking for the first-person voice, the writing still leaned on em-dashes and hyphen-as-punctuation constantly, a small tic that reads as machine-written the moment you notice it. I asked for it to stop entirely, in every file, not just new ones.

4. Where the credentials live. When we got to the AI copilot layer, I pointed it at an existing `.env` file from a different project rather than letting it invent a setup flow or ask me to paste a key into chat. It read the file, pulled just the one key it needed into this project's own gitignored `.env`, and never echoed the key value back into our conversation. I made the call on where the credential came from; it handled the mechanics of keeping it out of git.

5. Leaving Streamlit for a real frontend. The dashboard worked, but it looked and felt like a prototype: default widget styling, no real design system. I told it to drop Streamlit entirely and rebuild the interface in React and TypeScript with a proper backend API, not just restyle the existing script. That's a bigger ask than a tweak, and it meant

redoing the dashboard, the Playwright suite, and the CI workflow from a working baseline. Worth it: a shift supervisor's actual tool needs to look like something you'd trust your job to, not a demo.

6. Not trusting a flaky-looking test as "probably fine." After the React rebuild, one Playwright test (clicking a stand card, checking its charts) failed intermittently: passed once, failed with a `session closed` error the next run, failed differently the run after that. Along the way I'd also added an explicit `type="button"` to the interactive buttons, a legitimate fix on its own (buttons default to `type="submit"` in HTML), but I want to be honest that it wasn't actually the cause of the flakiness, and I nearly stopped there and called it fixed. Instead I wrote a small standalone script that launched a real browser, clicked the exact same element, and printed the actual before and after state directly, no test framework, no visual tool in between. That proved the click genuinely worked correctly on its own. Only after that clean, separate proof did I treat the remaining intermittent test failures as real environmental flakiness (a live network call to Anthropic's API plus a loaded dev machine) rather than a product bug, and fix the actual cause: increased timeouts and retries in the test config, not a suppressed assertion or a longer blind sleep.

Confidently wrong, and how it got caught

The rolling-mean baseline bug. When I first wired up the AI copilot's explanation layer, it told me, for a stand that was genuinely mid-failure: "*coolant pressure is also up.*" That's backwards. The whole failure signature is coolant pressure *dropping*. The model wasn't hallucinating a number; it correctly reported the +0.79 z-score it was given. The actual bug was upstream: I'd computed that z-score against the signal's own 60-minute rolling mean, and during an active fault that rolling mean is itself mid-collapse, so a noisy uptick within an ongoing drop can register as "above recent normal" even while the true value is crashing. I caught this myself, before it went any further, not because I eyeballed the sensor data and spotted it, but because I have a habit of actually reading model output for internal consistency rather than skimming it for vibes, and "up" didn't match what I knew the failure signature was supposed to look like. Fixed it by comparing against a stable, whole-training-period baseline instead of a window that moves during the exact event it's supposed to be measuring. Re-ran the same case live afterward: coolant pressure correctly showed -3.57 std devs.

The sys.path bug Playwright caught, that my own manual testing missed. This was during the Streamlit phase, before the React rebuild. I'd manually clicked through the dashboard in a real browser earlier and called it verified. It was verified, for the command I happened to use, `python -m streamlit run app/dashboard.py`. When I wrote the Playwright suite and pointed it at the *actual documented command* from the README, `streamlit run app/dashboard.py`, the app crashed on load with `ModuleNotFoundError: No module named 'app'`. Two different ways of launching the same script put different things on Python's import path, and I'd only ever tested the one that happened to work. This is the one that actually worried me a little: my own "I checked it in a browser, it's fine" was true and also wrong, because I'd tested a slightly different thing than what I told anyone else to run.

A sub-agent reported success on work it never did. While cleaning up the em-dash problem mentioned above, I delegated the mechanical part (rewriting 142 instances across 18 files) to a background sub-agent so it wouldn't eat up the main session's context. It came back with a status of "completed" and a written result claiming the work was done. When I actually checked, by re-searching the codebase for the character, every single instance was still there. The sub-agent had gotten confused partway through, tried to call a scheduling tool that didn't apply to its situation, and reported back as if that confusion were a finished task. I only caught it because I verify claims against the actual files instead of trusting a status message, which is the exact same instinct as the rolling-mean bug above: don't believe a report, check the artifact. I did the rewrite myself directly afterward.

The one I didn't catch myself: undownsampled charts, only found when I actually ran the app on my own machine. Every chart in the dashboard was being fed the full raw time series, about 13,000 points per stand per signal, with no downsampling. My own automated Playwright tests never caught this: the test only checked that the chart's container div was visible, which it was (it has a title and a border regardless of whether anything draws inside it), and separately, Playwright's own clean, resource-light test browser turned out to render even the full undownsampled

dataset without visibly failing. It took me actually opening the app on my own desktop, with the browser I actually use day to day and the other programs actually running on it, to see the real symptom: blank chart panels and a multi-second delay switching between stands. This is the clearest example in this whole log of a limit of AI-driven testing: an automated suite can prove a component exists and responds, but it cannot fully stand in for someone actually using the thing under realistic conditions. The fix was real (downsample to a target point count on the backend, always keeping every alert point so no flagged period visually disappears, sampling the rest), verified afterward with direct timing measurements: warm-state clicks went from failing to render at all down to under a second. I also went back and tightened the Playwright assertion itself, from "is the container visible" to "does the chart actually contain a rendered SVG path", specifically because the weaker check is what let this ship in the first place, documented in `tests/dashboard.spec.js`.

What I won't let it decide alone

Whether an anomaly triggers a real-world action. The dashboard explains and suggests; it never shuts anything down or dispatches anyone automatically. That's a design decision I made early and kept. A model that's right 99% of the time and wrong 1% of the time should not be the thing pulling a mill stand offline on its own, and I don't think that's a call an AI collaborator should get to talk me into relaxing for the sake of a flashier demo.

Where the actual alert threshold sits. Claude can compute the tradeoff curve; it can't tell me how much false-alarm fatigue a real shift crew would tolerate before they start ignoring the tool, because that's a judgment about people and workplace trust, not a number in a dataset.

Whether something is safe to publish. The confidentiality catch above is the clearest example. I'll take the flag seriously, but the actual "yes, push this" or "no, keep it local" is mine.

Whether a task actually got done. The sub-agent failure above is the clearest example of this one. A status of "completed" is a claim, not a fact, and I check the files, not the report.

Where it was 10x, where it slowed me down

10x: the sheer volume of working, tested code in one sitting. A synthetic data generator with a realistic non-linear failure ramp, a feature pipeline, a trained and evaluated anomaly detector, a FastAPI backend, a React frontend, an LLM-backed explanation layer, a Playwright suite, a GitHub Actions workflow, and this set of docs. Solo, without an AI collaborator doing the typing, this is realistically a multi-week effort for a single person around a day job, not because any one piece is hard, but because there are a lot of pieces and switching between Python, ML evaluation, frontend, and CI config context is normally where time disappears.

Where it slowed me down: two things. The editor's type checker kept flagging things inside the code (pandas method calls, sklearn constructor arguments) as errors that weren't actually errors at runtime, stale type-stub noise, not real bugs. Every one of those needed a second look to confirm it was noise and not something real. And the sub-agent failure above cost real time: a delegated task that silently didn't happen is worse than one that visibly fails, because it looks finished until you check.

Addendum: closing the model comparison gap

A fair question surfaced after the original submission: did I ever actually test IsolationForest against other unsupervised models, or just reason my way to it and stop? Honest answer: I never tested it. `docs/architecture.md`'s original "what I considered and rejected" section explained why a supervised deep model was skipped, which was sound reasoning,

but it quietly implied a rigor around the model choice itself that wasn't actually there. IsolationForest was picked and never benchmarked against anything in its own weight class.

I built `notebooks/model_comparison.ipynb` to close that gap directly. It reuses the actual production ingestion and feature code (`app/features.py` , `app/labels.py`), so results transfer to the real pipeline instead of testing a simplified stand in. It trains a naive baseline plus four real candidates (IsolationForest, LocalOutlierFactor, OneClassSVM, EllipticEnvelope) and grades every one of them with the exact same lead time and false alarm logic `app/evaluate.py` already used, then executed the notebook for real (via `nbconvert --execute`) rather than hand typing plausible looking numbers into markdown.

The result was not close: LocalOutlierFactor beat the shipped IsolationForest on both axes that matter at once, zero false alarms versus about 0.8%, and roughly ten times more warning time before failure. I updated `app/detector.py` to train LocalOutlierFactor in production and re ran `python -m app.evaluate` to confirm the deployed numbers matched the notebook exactly at the time (they did: 11/11 detected, 0.00% false alarm rate, 562 minute median lead time in both places), rather than trusting the notebook's numbers and assuming the production code would behave the same. Those exact numbers changed again in the next addendum below, for good reason.

Two things I want to be direct about. First, this is the kind of gap that's easy to paper over with confident sounding prose instead of an actual experiment, and I did exactly that the first time around. Second, "beats IsolationForest on synthetic data" is not the same claim as "is the right model for a real mill." Both the notebook and `docs/architecture.md` now say so explicitly: the synthetic failure signature is a strong, learnable signal by construction, all five candidates caught every failure, and the real world comparison would need to be re run against actual historian data before this result is trusted past prototype stage.

Addendum: a user question exposed a real model blind spot, and fixing it surfaced a deeper bug

The user asked a plain question about the fleet dashboard: are the numbers on the landing page random, or real? Answering it honestly required actually checking, not asserting. They're real (the last row of the held out test window's static CSV, not random and not live), but checking also surfaced something I hadn't caught: three stands showed vibration readings around 5x their normal baseline while still labeled "Normal."

Digging into STAND-01 specifically: it failed at 18:01 that day. Six hours later, at 23:59, vibration was still 8 to 10.6 mm/s against a 2.54 baseline, clearly still broken, but `anomaly_score` had settled to about 1.0 to 1.1, under the 1.299 alert threshold. The model wasn't malfunctioning, it was doing exactly what LocalOutlierFactor does: measuring local density. Six hours into a sustained failure, the recent minutes all look like each other, so they become each other's "normal" neighborhood and the score drops, even though the raw reading never recovered. This is the flip side of the exact property that made LOF win the bake off in the first place (catching subtle onset early via local density), so it wasn't something a different threshold would fix.

I added a backstop in `app/detector.py` : independent of the model entirely, if any raw signal sits more than 3.5 standard deviations from the STABLE (whole training period) baseline for 20 consecutive minutes, force an alert regardless of what the model says. First test run: still 11/11 detected, same lead times, but false alarm rate jumped from 0.00% to 4.18%. My first instinct was to just raise the z threshold until the number looked good again. I didn't, because that would have been tuning away a symptom without knowing the cause, the same mistake the model comparison gap above was about.

Checked the actual flagged rows instead. Every one of them was in the 24 to 48 hour range after a stand's OWN failure, not before its next one, times like STAND-01 at 13:22 the day after a 13:21 failure, still visibly broken, vibration still 8.7 to 11.3. The bug wasn't in the backstop. It was in `app/labels.py` : `time_to_failure_min` only ever looked forward to

the next failure. A row could be 28 hours from the next failure and still be sitting in the unrecovered aftermath of the previous one (the generator holds a stand at its failed values for the rest of the day once it fails), and the existing "normal" filter counted it as clean baseline anyway. That gap existed before the backstop; LocalOutlierFactor's own local-density blind spot had just been quietly absorbing it without complaint, which is exactly why nobody had noticed it before.

Fixed it at the source: added `time_since_failure_min` (a backward `merge_asof`, symmetric to the existing forward one) and a shared `is_clean_normal()` helper requiring distance from a failure in BOTH directions, used for both `train_normal` and the false-positive evaluation, not just the backstop. Re ran the full pipeline: false alarm rate came back down to 0.05%, essentially the handful of genuine edge cases the backstop is supposed to catch, not noise. The real surprise was that lead time improved too: median went from 562 to 687 minutes. Removing ~55,000 contaminated recovery-period rows from `train_normal` gave the model a cleaner definition of "normal" to train against, which is not something I set out to fix, it fell out of fixing the labeling bug correctly instead of papering over the symptom.

What I'd flag for anyone reading this closely: I did not re run `notebooks/model_comparison.ipynb` after this fix. The labeling bug it inherited would have affected all five candidate models symmetrically (cleaner training data helps all of them, not just LocalOutlierFactor), so I don't expect it to change which model wins, but that is a reasoned judgment call, not a verified fact, and it should be re run before treating the original bake off numbers as current.

Addendum: building the live tier, and everything that broke on the way there

The dashboard moved from a single static, already scored CSV to an actual live feed, a manual degradation simulator, and a free text fleet copilot (full description in `docs/architecture.md`'s addendum). None of that shipped clean the first time.

The override that saved me a wasted hour: don't retrain on a date change. After I changed `data/generate_synthetic_data.py` to anchor its window to end yesterday instead of a fixed date, my own next move was going to be re running `python -m app.evaluate` to retrain, because "new dates" read as "new data" in my head. The user stopped me and asked why retraining was even necessary if this was just for plotting. Good question, and the honest answer is it wasn't necessary at all: the generator uses a fixed random seed, so different calendar labels produce byte for byte identical signal sequences, and the trained model only ever sees engineered rolling stats, never a date. Retraining would have been pure wasted compute that also risked quietly shifting the documented 11/11 detection and 0.05% false alarm numbers for no real reason. This is the same category as the confidentiality catch at the top of this doc, someone else noticing I was about to do something I hadn't actually thought through, except this time it caught me wasting effort instead of catching a policy violation.

Confidently wrong, caught by my own test, not by guessing. When I built the fleet wide query functions in `app/fleet_tools.py`, I ran a direct standalone test of `get_current_reading` before wiring anything else to it, and it threw `AttributeError: 'DataFrame' object has no attribute 'stand_id'` on a completely empty frame. Tracing it back: `live_feed.py`'s `get_scored()` still checked `if _buffer is None`, left over from an earlier version where the buffer really could be `None`. I'd since changed it to always be a real (possibly empty) DataFrame to satisfy the type checker, and updated the OTHER place that needed an emptiness check, but missed this one. `is None` can now never be true, so the function silently returned an empty, columnless frame instead of ever seeding real data. It never showed up in the actual running app, because the server's own startup task ticks once immediately and papers over it, which is exactly why it's worth naming: a bug that only manifests in a code path the happy path never exercises is still a real bug, and I only found it because I tested the new piece in isolation instead of only ever testing it through the full running app.

A guardrail that was too strict, until a real question hit it. I told the fleet copilot to never guess or invent a stand a tool didn't return, meaning to stop it hallucinating readings. First real question from the user, "how often has A failed,"

and instead of answering it asked which of the six stands they meant. Technically obedient to what I'd written, actually useless, a shift supervisor saying "stand A" obviously means something. The instruction I'd written didn't distinguish between inventing a number (bad) and resolving an informal name to a real id (a parsing step, not data invention). Added an explicit rule mapping letters and bare numbers to STAND-01 through STAND-06 and telling it to only ask for real clarification when a reference genuinely can't map to any of the six. Same underlying lesson as the rolling mean bug earlier in this doc: a correctly followed instruction can still produce the wrong outcome if the instruction itself didn't anticipate the actual case in front of it.

The house style problem showed up again, in a place I don't hand write. This doc's very first override entry is about fighting em-dashes and hyphen-as-punctuation out of my own writing. They showed up again anyway, this time inside the fleet copilot's actual runtime output, text I never touch by hand, generated fresh on every question. A prompt instruction telling the model not to use them didn't reliably stop it. Added a small regex that strips dash-as-punctuation from the model's output after the fact, on top of the prompt instruction, not instead of it, because telling a model not to do something once is not the same guarantee as it actually never doing it.

Slow the first time, and I almost shipped the slow version. Scoring the full 45 day historical set through LocalOutlierFactor for the new date range filter took about 100 seconds the first time I measured it, an obvious non-starter inside a live request. My first fix cached that result, but a bug in the "does this date range even need the historical set" check meant even the plain Today filter, which never touches historical data at all, was triggering the full 100 second computation on a fresh server. Caught it by actually timing the request instead of assuming the cache fixed everything, found the check was comparing the requested range against the live buffer's own window instead of the historical set's real date bounds, and fixed it to check the real bounds directly.

Addendum: the copilot answered confidently, and the answer was still weak

The user asked the fleet copilot which stand was next to alert. It answered with the stand carrying the highest current anomaly score, 1.2 against a 1.288 threshold, presented as if that settled the question. The user called this out directly: a snapshot score close to a threshold is not evidence something is *about* to cross it, that score could be rising, falling, or sitting flat, and the answer never said which. This is the same shape of problem as the rolling mean bug earlier in this doc, a number correctly reported that still produced a misleading sentence, except this time the number itself was simply the wrong evidence for the question being asked, not miscomputed.

The actual gap: every tool available to the copilot (`app/fleet_tools.py`) only ever answered "what is true right now." Nothing exposed direction of travel, so "which stand is likely to alert soon" had no real evidence to call on and the model filled the gap with the nearest number that sounded relevant. Added `get_recent_trend`, comparing a stand's average anomaly score over the first versus second half of a recent window to classify it as rising, falling, or flat, and told the system prompt explicitly that a current score alone is not an answer to a forward looking question, call this first and answer from actual direction, not just whichever stand scores highest at this instant.

Testing that fix directly surfaced two more real bugs, not visible from reading the code, only from actually running the same question repeatedly:

A stall that only showed up about a third of the time. Roughly one in three runs, the model responded with something like "I'll check the current readings and trends across all stands" and then stopped, never actually calling anything, handing the supervisor a sentence about a plan instead of an answer. A single test would have looked fine and hidden this completely. Added a check for that specific "I'll / I will / let me" self-narrating pattern on any response that made no tool calls, and if it fires, the code feeds the model back a direct nudge to actually call the tool instead of describing that it's about to, rather than showing the supervisor a stall.

A cut-off response that silently dropped real tool calls. Tracing one of the stall-looking failures directly (printing every message and every response's stop reason, not guessing from the error text) showed the model had genuinely

started calling `get_recent_trend` on several stands in one turn, a text preamble plus seven tool calls, and got cut off mid-response by hitting the 300 token reply limit before finishing. My code was deciding whether tool calls happened by checking `response.stop_reason == "tool_use"`, which was false here (the real reason was `max_tokens`), so it discarded seven live, complete tool call requests and returned the leftover preamble text as if it were the final answer. Worse, on the next question those orphaned tool call ids got resent without their results and the API flatly rejected the request. Fixed both causes, not just the symptom: raised the token budget for this specific multi-tool-call path to 1024, and rewrote the branch to check whether the response actually contains tool_use content instead of trusting stop_reason alone, so a truncated-but-real batch of tool calls gets executed and answered instead of thrown away.